

Python - Functions & Return Values

Grade 3 | Coding Adventure

1. What is a Function?

A **function** is a reusable block of code that helps us do a specific job. Instead of writing the same lines of code over and over again, we can bundle them into a function and use it whenever we want! It helps keep our code neat, organized, and saves us lots of repeating text.



Think of it like a Machine:

1. Input

You give the machine something to work with.

2. Process

The machine does a specific job or calculation.

3. Output

The machine gives something back to you!



Real-Life Example: A Juice Blender

Imagine using a kitchen juice blender machine:

- **Input:** You put whole oranges into the blender.
- **Process:** The machine spins its blades and squeezes them up.
- **Output:** You get delicious, fresh orange juice back!

2. Creating & Defining a Function

In Python, we use a special magic word called **def** to create a function. **def** stands for "define".

```
def say_hello():  
    print("Hello, friend!")
```

Let's break down how this works:

- **def** tells Python: "Hey! We are starting to define a brand-new function!"
- **say_hello** is the name of our function. You can name it almost anything!
- The brackets **()** are followed by a colon **:** which officially starts the inside body of the function.
- The **indented code** (the pushed-in line with spaces) tells Python exactly which commands belong inside this function box.

3. Calling a Function

Just creating a function doesn't make it run! It is like building a machine but leaving it turned off. To make the code inside execute, you have to **call it by its name** with brackets **()** attached:

```
def say_hello():
    print("Hello, friend!")

# This line actually turns on and runs the function machine!
say_hello()

# Output on screen: Hello, friend!
```

Try It Yourself Challenge 1!

Type the following code into your Python window. What do you see appear on the screen?

```
def greet():
    print("Namaste!")

print("Welcome to coding!")
greet()
```

4. Parameters (Giving Inputs)

Sometimes, we want to give our function some extra information to work with. We do this using **Parameters**. Think of a parameter like a blank variable slot inside the brackets that gets filled in when you run the function!

```
def greet(name):
    print("Hello, " + name + "!")
```

Now, we can use the exact same function with different inputs to get unique results every time:

```
greet("Aarav")
greet("Priya")

# Output on screen:
# Hello, Aarav!
# Hello, Priya!
```

+ Using Multiple Parameters

Functions can accept more than one input slot! Just separate your parameters using a comma , :

```
def add(a, b):
    print(a + b)

add(3, 5)      # Output: 8
add(10, 20)   # Output: 30
```

5. Understanding the 'return' Keyword

What if your function calculates an answer and needs to hand it back to your main code program? We use the **return** keyword!

What is the difference?

- **print()** simply throws text onto the display screen for you to look at.
- **return** safely sends a value **BACK** from the function machine so your program can save it, change it, or use it later!

```
def add(a, b):
    return a + b

# The function calculates 4 + 6, gives back 10, and saves it in 'result'
result = add(4, 6)
print(result)
```

```
# Output: 10
```

You can use returned answers anywhere in your code, or even plug them directly into other functions:

```
def double(n):
    return n * 2

x = double(7)    # x becomes 14
print(x)         # Output: 14
print(double(3)) # Output: 6
```

Interactive Practice Lab

Practice Question 1: Write a function called `square` that takes a number and returns it multiplied by itself. Guess what prints out below!

```
def square(n):
    return n * n

print(square(5)) # What prints? Hint: 5 * 5 = ____
print(square(9)) # What prints? Hint: 9 * 9 = ____
```

Practice Question 2: Look at this function called `is_even`. Remember, the `%` sign gives you the mathematical remainder left over after dividing!

```
def is_even(n):
    return n % 2 == 0

print(is_even(4)) # Prints: True (4 divided by 2 has 0 remainder)
print(is_even(7)) # Prints: False (7 divided by 2 has a remainder of 1)
```

★ Key Takeaways (Your Coding Superpowers!)

- ✨ **def** sets up and creates a new function box.
- ✨ Call a function by writing its name followed by brackets **()**.
- ✨ **Parameters** are the inputs passed inside the brackets.
- ✨ **return** shoots a finished answer value back to your main code.
- ✨ Functions make your code tidy, reusable, and super neat!

🎉 Great Job! You are becoming a true Python Pro! Keep Practising! 🚀